



UNIMORE
UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA

Most of the slides are taken from

Eric Freeman, Elisabeth Robson, Bert Bates,
Kathy SierraHead First Design Patterns:
Building Extensible and Maintainable Object-
Oriented Software, O'Reilly Media 2020

Design Pattern Singleton and Command Patterns

Francesco Guerra
francesco.guerra@unimore.it

Singleton Pattern

- ▶ It creates one-of-a-kind objects for which there is only one instance.
 - ▶ The Singleton is the simplest path in terms of its class diagram;
- ▶ Despite its simplicity from a class design perspective, its implementation can present critical issues.
- ▶ Motivation
 - ▶ There are many objects we only need one of:
 - ▶ thread pools, caches, dialog boxes,
 - ▶ objects that handle preferences and registry settings, objects used for logging
 - ▶ objects that act as device drivers to devices like printers and graphics cards.

The Singleton Pattern ensures a class has only one instance, and provides a global point of access to it

Instantiate an object

- ▶ So as long as we have a public class, I can always instantiate an object one or more times?

```
new MyObject();
```

- ▶ If it's not a public class, it could be protected and only classes in the same package can instantiate it.

- ▶ But they can still instantiate it more than once.

- ▶ What about?

```
public MyClass {  
    private MyClass() {}  
}
```

- ▶ This is legal but class that can't be instantiated.

- ▶ Is there ANY object that could use the private constructor?

Instantiate an object

- ▶ Can I use a static variable to instantiate a class?

```
public MyClass {  
    private MyClass() {}  
    public static MyClass getInstance() {  
        return new MyClass();  
    }  
}
```

```
MyClass.getInstance();
```

- ▶ How can I modify the code so that only a single object will be instantiated?

The classic Singleton Pattern implementation

```
public class Singleton {  
    private static Singleton uniqueInstance;  
  
    // other useful instance variables here  
  
    private Singleton() {}  
  
    public static Singleton getInstance() {  
        if (uniqueInstance == null) {  
            uniqueInstance = new Singleton();  
        }  
        return uniqueInstance;  
    }  
    // other useful methods here  
}
```

Motivating example

- We have to manage a controller for cauldron used in the production of Parmigiano Reggiano cheese

```

public class Cauldron{
    private boolean empty;
    private boolean boiled;

    public Cauldron() {
        empty = true;
        boiled = false;
    }
    public void fill() {
        if (isEmpty()) {
            empty = false;
            boiled = false;
            // fill the cauldron
            with a milk mixture
        }
    }
    public void removed() {
        if (!isEmpty() && isBoiled()) {
            // remove the boiled milk
            empty = true;
        }
    }

    public void boil() {
        if (!isEmpty() && !isBoiled()) {
            // bring the contents to
            a boil
            boiled = true;
        }
    }

    public boolean isEmpty() {
        return empty;
    }

    public boolean isBoiled() {
        return boiled;
    }
}

```

Using a Singleton

```
public class Cauldron{
    private boolean empty;
    private boolean boiled;
    private static Cauldron uniqueInstance;
    private Cauldron() {
        empty = true;
        boiled = false;
    }
    public static Cauldron getInstance() {
        if (uniqueInstance == null) {
            uniqueInstance = new Cauldron();
        }
        return uniqueInstance;
    }
    public void fill() {
        if (isEmpty()) {
            empty = false;
            boiled = false;
        }
    }
    // rest of Cauldron code...
}
```



Managing multiple threads

- ▶ In case of multiple threads we can have two instantiations of the same class

Thread 1

```
public static Cauldron  
getInstance() {
```

```
    if (uniqueInstance == null) {
```

```
        uniqueInstance =  
            new Cauldron();
```

```
    return uniqueInstance
```

Thread 2

```
public static Cauldron  
getInstance() {
```

```
    if (uniqueInstance == null) {
```

```
        uniqueInstance =  
            new Cauldron();
```

```
    return uniqueInstance
```


Dealing with multithreading

```
public class Singleton {
    private static Singleton uniqueInstance;

    // other useful instance variables here

    private Singleton() {}

    public static synchronized Singleton getInstance() {
        if (uniqueInstance == null) {
            uniqueInstance = new Singleton();
        }
        return uniqueInstance;
    }

    // other useful methods here
}
```

- ▶ By adding the `synchronized` keyword to `getInstance()`, we force every thread to wait its turn before it can enter the method.
 - ▶ Synchronizing a method can decrease performance by a factor of 100

How to improve multithreading

- ▶ Do nothing if the performance of `getInstance()` isn't critical to the application.
- ▶ Move to an eagerly created instance rather than a lazily created one.

```
public class Singleton {  
    private static Singleton uniqueInstance = new Singleton();  
  
    private Singleton() {}  
  
    public static Singleton getInstance() {  
        return uniqueInstance;  
    }  
}
```

We rely on the JVM to create the unique instance of the Singleton when the class is loaded.

The JVM guarantees that the instance will be created before any thread accesses the static `uniqueInstance` variable.

How to improve multithreading (2)

- Use “double-checked locking” to reduce the use of synchronization in `getInstance()`.

```
public class Singleton {  
    private volatile static Singleton uniqueInstance;  
  
    private Singleton() {}  
  
    public static Singleton getInstance() {  
        if (uniqueInstance == null) {  
            synchronized (Singleton.class) {  
                if (uniqueInstance == null) {  
                    uniqueInstance = new Singleton();  
                }  
            }  
        }  
        return uniqueInstance;  
    }  
}
```

The `volatile` keyword ensures that multiple threads handle the `uniqueInstance` variable correctly when it is being initialized to the Singleton instance.

Java Implementation with enum

```
1 public enum SingletonEnum {
2     INSTANCE;
3
4     int value;
5
6     public int getValue() {
7         return value;
8     }
9
10    public void setValue(int value) {
11        this.value = value;
12    }
13 }
```

<https://dzone.com/articles/java-singletons-using-enum>

```
1 public class EnumDemo {
2
3     public static void main(String[] args) {
4         SingletonEnum singleton = SingletonEnum.INSTANCE;
5
6         System.out.println(singleton.getValue());
7         singleton.setValue(2);
8         System.out.println(singleton.getValue());
9     }
10 }
```



Singleton

- singleton : Singleton

- Singleton()

+ getInstance() : Singleton

Singleton Pattern defined

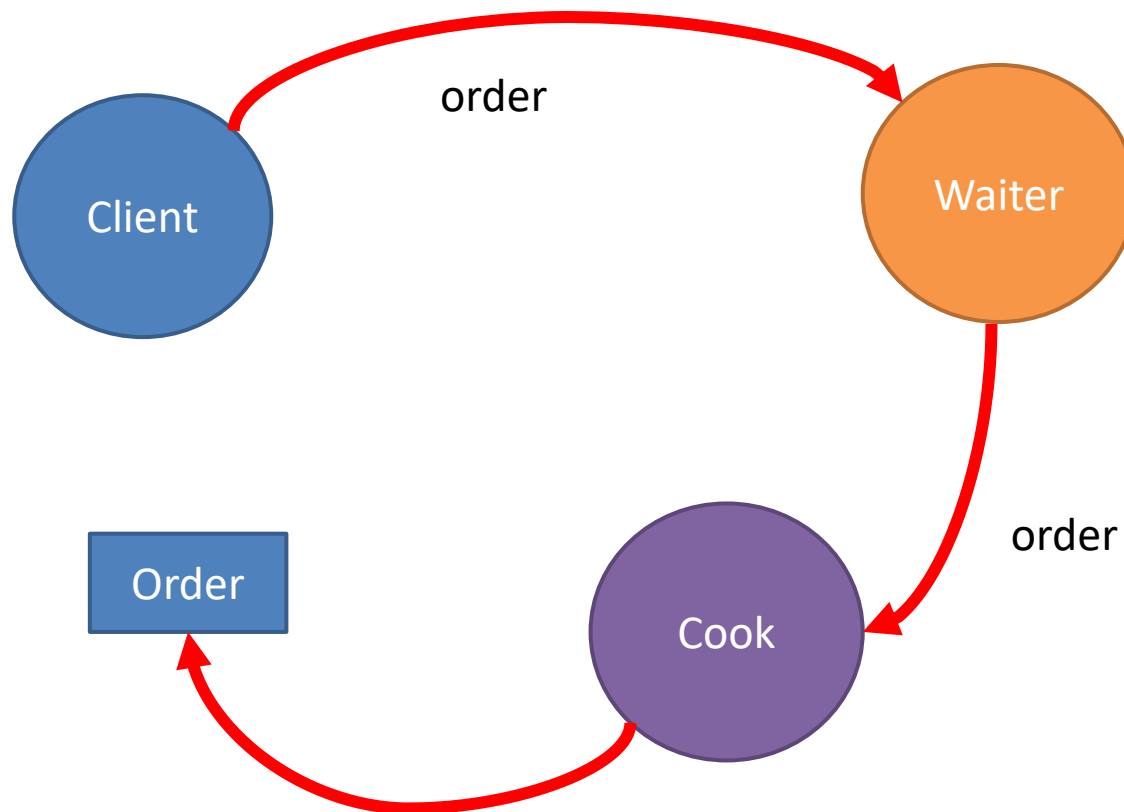
- ▶ The Singleton Pattern ensures you have
 - ▶ at most one instance of a class in your application.
 - ▶ a global access point to that instance. □
- ▶ Java's implementation of the Singleton Pattern makes use of a private constructor, a static method combined with a static variable.
- ▶ Examine your performance and resource constraints and carefully choose an appropriate Singleton implementation for multithreaded applications.
 - ▶ Be careful if you are using multiple class loaders; this could defeat the Singleton implementation and result in multiple instances.

The Command Pattern

- ▶ The Command Pattern allows us to encapsulate method invocation.
 - ▶ By encapsulating method invocation, we can crystallize pieces of computation so that the object invoking the computation doesn't need to worry about how to do things.
 - ▶ We can also do some smart things with these encapsulated method invocations, like save them away for logging or reuse them to implement undo in our code.

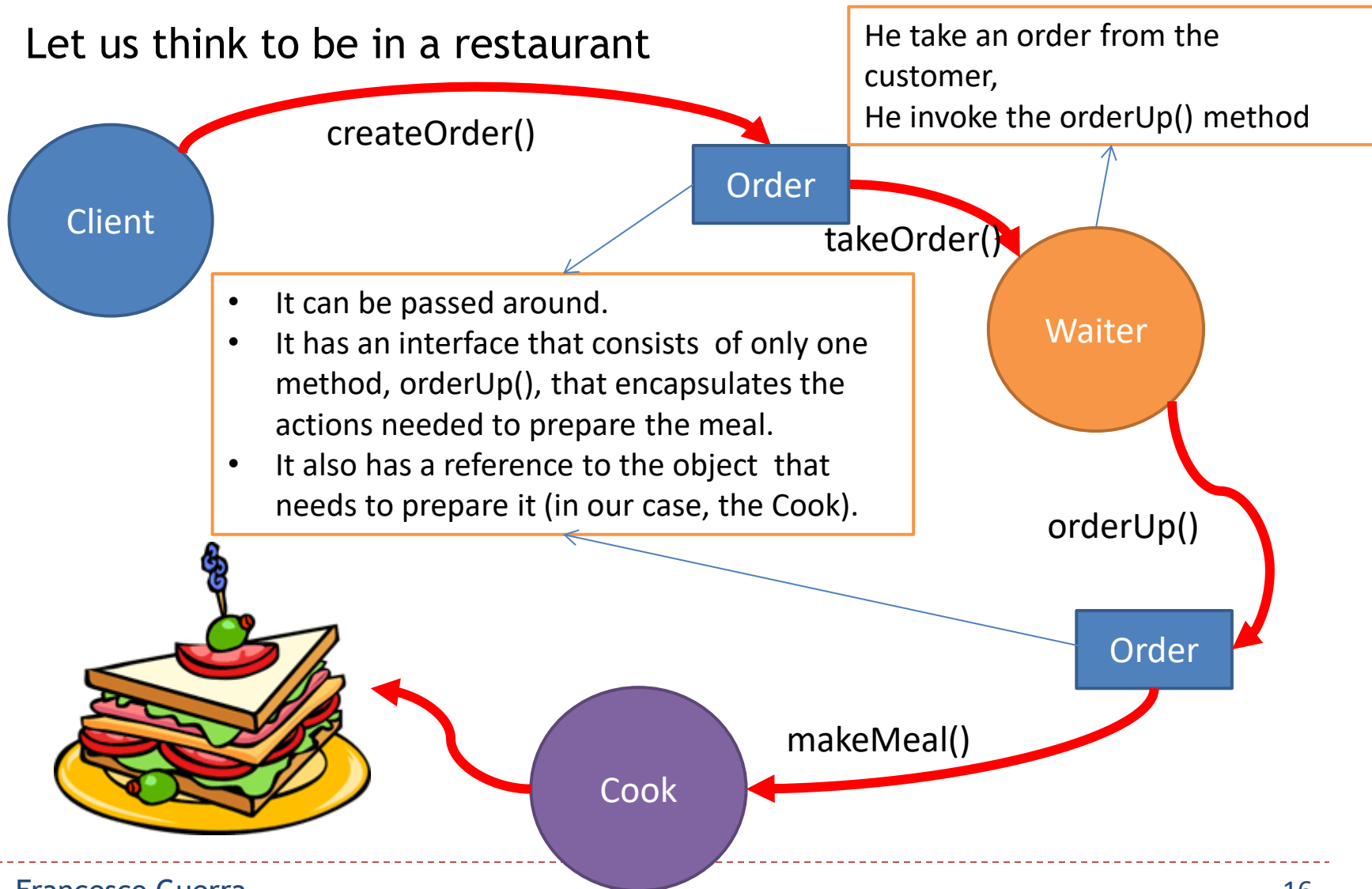
The Command Pattern - motivation

- Let us think to be in a restaurant

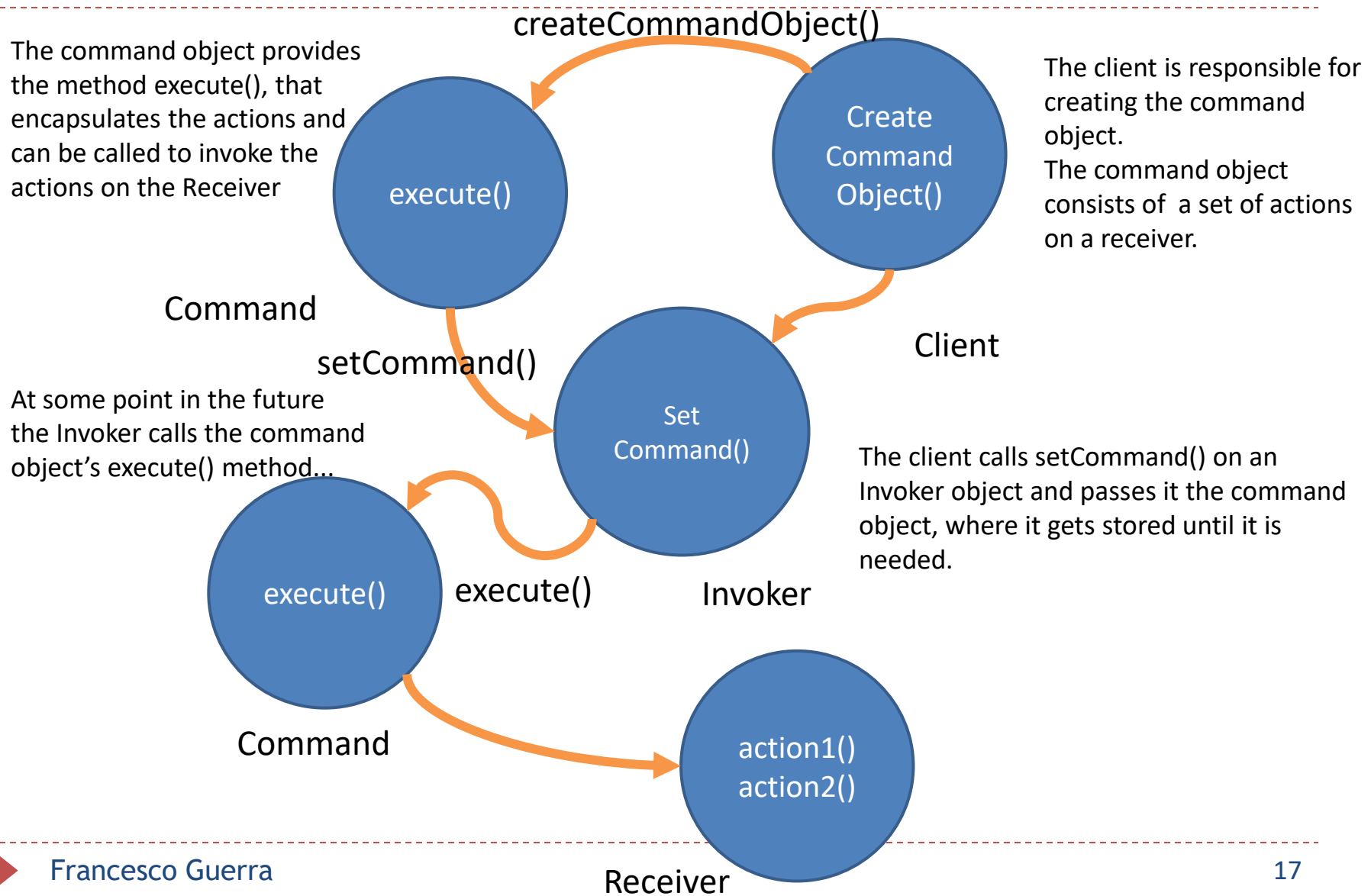


The Command Pattern - overview

- Let us think to be in a restaurant

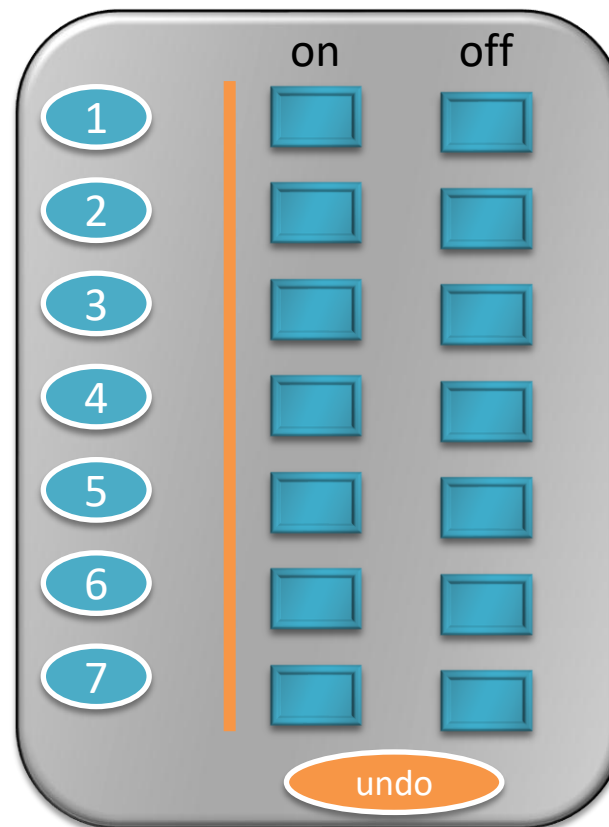


In other words



Example

- ▶ You have to implement the code for handling a universal remote control for managing several devices in your home (e.g. light and TV control, ...)



Implementing the device

```
public interface Command {  
    public void execute();  
}
```

Implementing the Command interface

Implementing a command to turn a light on

```
public class LightOnCommand implements Command {  
    Light light;  
  
    public LightOnCommand(Light light) {  
        this.light = light;  
    }  
  
    public void execute() {  
        light.on();  
    }  
}
```

A command has to be created for each device
and for each function we can enable

Implementing a simple Remote Control

```
public class SimpleRemoteControl {
    Command slot;
    public SimpleRemoteControl() {}

    public void setCommand(Command command) {
        slot = command;
    }
    public void buttonWasPressed() {
        slot.execute();
    }
}
```

client

```
public class RemoteControlTest {
    public static void main(String[] args) {
        SimpleRemoteControl remote = new SimpleRemoteControl();
        Light light = new Light();
        LightOnCommand lightOn = new LightOnCommand(light);

        remote.setCommand(lightOn);
        remote.buttonWasPressed();
    }
}
```

invoker

Implementing the “big” Remote Control

```
public class RemoteControl {
    Command[] onCommands;
    Command[] offCommands;

    public RemoteControl() {
        onCommands = new Command[7];
        offCommands = new Command[7];

        Command noCommand = new NoCommand();
        for (int i = 0; i < 7; i++) {
            onCommands[i] = noCommand;
            offCommands[i] = noCommand;
        }
    }
}
```

Implementing the “big” Remote Control

```
public void setCommand(int slot, Command onCommand, Command offCommand) {
    onCommands[slot] = onCommand;
    offCommands[slot] = offCommand;
}

public void onButtonWasPushed(int slot) {
    onCommands[slot].execute();
}

public void offButtonWasPushed(int slot) {
    offCommands[slot].execute();
}

public String toString() {
    StringBuffer stringBuffer = new StringBuffer();
    stringBuffer.append("\n----- Remote Control ----- \n");
    for (int i = 0; i < onCommands.length; i++) {
        stringBuffer.append("[slot " + i + "] " +
onCommands[i].getClass().getName()
        + " " + offCommands[i].getClass().getName() + " \n");
    }
    return stringBuffer.toString();
}
}
```





Implementing the Commands

```

public class LightOffCommand
implements Command {
    Light light;

    public LightOffCommand(Light
light) {
        this.light = light;
    }

    public void execute() {
        light.off();
    }
}
  
```

```

public class TVOn implements
Command {
    TV tv;

    public TVOn (TV tv) {
        this.tv = tv;
    }

    public void execute() {
        tv.on();
        tv.setChannel(1);
        tv.setVolume(11);
    }
}
  
```

Light and TV

```
public class TV {
    String location = "";

    public TV(String location) {
        this.location = location;
    }

    public void on(){
        System.out.println(location + " TV is on!");
    }
    public void off(){
        System.out.println(location + " TV is
off!");
    }
}
```

```
public class Light {
    String location = "";

    public Light(String location) {
        this.location = location;
    }

    public void on(){
        System.out.println(location + " light is on!");
    }
    public void off(){
        System.out.println(location + "light is off!");
    }
}
```



Setting the Remote Control

```
public class RemoteLoader {  
  
    public static void main(String[] args) {  
        RemoteControl remoteControl = new RemoteControl();  
        Light livingRoomLight = new Light("Living Room");  
        Light kitchenLight = new Light("Kitchen");  
        TV tv = new TV("Living Room");  
        LightOnCommand livingRoomLightOn =  
            new LightOnCommand(livingRoomLight);  
        LightOffCommand livingRoomLightOff =  
            new LightOffCommand(livingRoomLight);  
        LightOnCommand kitchenLightOn =  
            new LightOnCommand(kitchenLight);  
        LightOffCommand kitchenLightOff =  
            new LightOffCommand(kitchenLight);  
  
        TVOnCommand tvOn= new TVOnCommand(tv);  
        TVOffCommand tvOff = new TVOffCommand(tv);  
  
        remoteControl.setCommand(0, livingRoomLightOn,  
livingRoomLightOff);  
        remoteControl.setCommand(1, kitchenLightOn, kitchenLightOff);  
        remoteControl.setCommand(2, tvOn, tvOff);  
    }  
}
```

Setting the Remote Control(2)

```
System.out.println(remoteControl);

remoteControl.onButtonWasPushed(0);
remoteControl.onButtonWasPushed(1);
remoteControl.onButtonWasPushed(2);
remoteControl.offButtonWasPushed(2);
remoteControl.offButtonWasPushed(1);
remoteControl.offButtonWasPushed(0);
    }
}
```

NoCommand

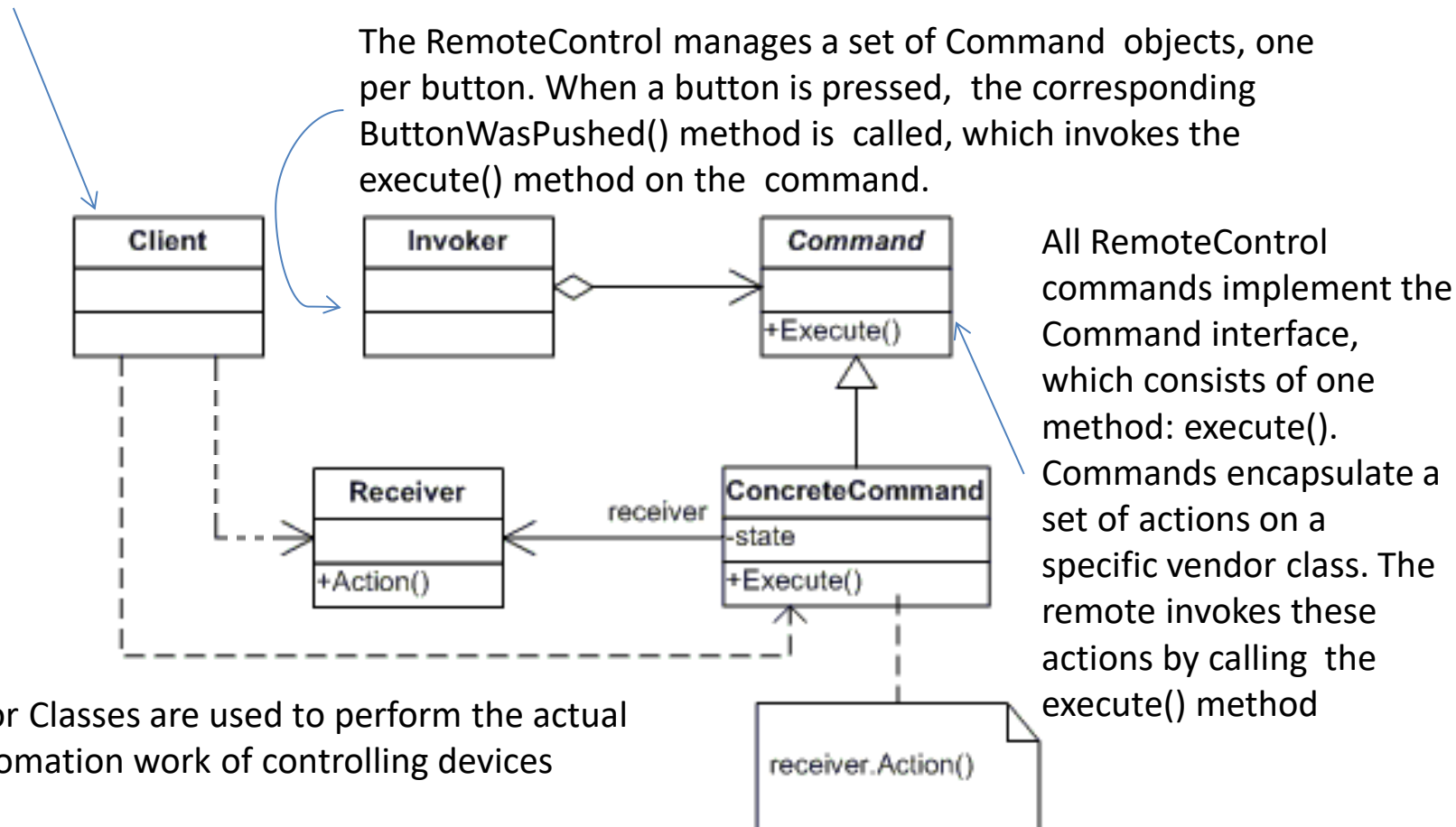
```
public class NoCommand implements Command {  
    public void execute() { }  
}
```

- ▶ The NoCommand object is an example of a null object.
 - ▶ A null object is useful when you don't have a meaningful object to return, and yet you want to remove the responsibility for handling null from the client.
- ▶ You'll find uses for Null Objects in conjunction with many Design Patterns
 - ▶ sometimes you'll even see Null Object listed as a Design Pattern.

The Client (Remote Loader) creates a number of Command objects. These are loaded into the slots of the Remote Control.

The UML Diagram

The RemoteControl manages a set of Command objects, one per button. When a button is pressed, the corresponding ButtonWasPushed() method is called, which invokes the execute() method on the command.



The Vendor Classes are used to perform the actual home-automation work of controlling devices

Using the Command Interface, we implement each action that can be invoked by pressing a button on the remote with a Command object. The Command object holds a reference to an object and implements an execute method that calls one or more methods on that object.

... and the UNDO Button?

- ▶ When commands support undo, they have an `undo()` method that mirrors the `execute()` method.

```
public interface Command {  
    public void execute();  
    public void undo();  
}
```

- ▶ We have to modify the concrete command

```
public class LightOnCommand implements Command {  
    Light light;  
    public LightOnCommand(Light light) {  
        this.light = light;  
    }  
    public void execute() {  
        light.on();  
    }  
    public void undo() {  
        light.off();  
    }  
}
```

... and the UNDO Button? (2)

- ▶ We have also to modify the remote control

```
public class RemoteControlWithUndo {
    Command[] onCommands;
    Command[] offCommands;
    Command undoCommand;
    ...
    public void onButtonWasPushed(int slot) {
        onCommands[slot].execute();
        undoCommand = onCommands[slot];
    }

    public void offButtonWasPushed(int slot) {
        offCommands[slot].execute();
        undoCommand = offCommands[slot];
    }

    public void undoButtonWasPushed() {
        undoCommand.undo();
    }
}
```

Implementing Macro-Commands

- ▶ A macro command is a command which executes one or more other commands

```
public class MacroCommand implements Command {  
    Command[] commands;  
  
    public MacroCommand(Command[] commands) {  
        this.commands = commands;  
    }  
  
    public void execute() {  
        for (int i = 0; i < commands.length; i++) {  
            commands[i].execute();  
        }  
    }  
}
```

```
Command[] allOn = { lightOn, stereoOn, tvOn, hottubOn};  
MacroCommand allOnMacro = new MacroCommand(allOn);
```

More uses of the Command Pattern: queuing requests

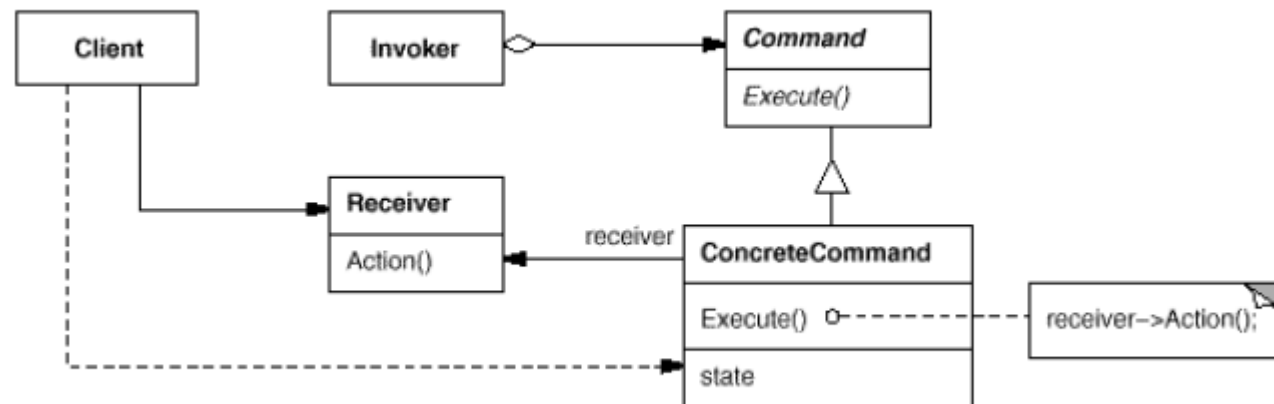
- ▶ Commands give us a way to package a piece of computation (a receiver and a set of actions) and pass it around as a first-class object.
 - ▶ The computation may be invoked after some client application creates the command object.
 - ▶ Useful applications such as schedulers, thread pools, and job queues,...
- ▶ Imagine a job queue:
 - ▶ you add commands to the queue on one end, and on the other end sits a group of threads.
 - ▶ Threads run the following script:
 - ▶ they remove a command from the queue,
 - ▶ call its execute() method,
 - ▶ wait for the call to finish,
 - ▶ then discard the command object,
 - ▶ retrieve a new one.
- ▶ The job queue classes are decoupled from the objects doing the computation.
- ▶ The job queue objects just retrieve commands and call execute().

More uses of the Command Pattern: logging requests

- ▶ The semantics of some applications require that we log all actions and be able to recover after a crash by reinvoking those actions.
- ▶ The Command Pattern can support these semantics with the addition of two methods: `store()` and `load()`.
 - ▶ As we execute commands, we store a history of them on disk.
 - ▶ When a crash occurs, we reload the command objects and invoke their `execute()` methods in batch and in order.
- ▶ By using logging, we can save all the operations since the last checkpoint, and if there is a system failure, apply those operations to our checkpoint.

The Command Pattern defined

- ▶ The Command Pattern encapsulates a request as an object, thereby letting you parameterize other objects with different requests, queue or log requests, and support undoable operations.



Key Points

- ▶ The Command Pattern decouples an object making a request from the one that knows how to perform it.
- ▶ A Command object is at the center of this decoupling and encapsulates a receiver with an action (or set of actions). An invoker makes a request of a Command object by calling its execute() method, which invokes those actions on the receiver.
- ▶ Invokers can be parameterized with Commands, even dynamically at runtime.
- ▶ Commands may support undo by implementing an undo method that restores the object to its previous state before the execute() method was last called. □
- ▶ Macro Commands are a simple extension of Command that allow multiple commands to be invoked.
 - ▶ In practice, it is not uncommon for “smart” Command objects to implement the request themselves rather than delegating to a receiver.
 - ▶ Commands may also be used to implement logging and transactional systems.



References

- ▶ Eric Freeman, Elisabeth Robson, Bert Bates, Kathy Sierra: Head First Design Patterns: Building Extensible and Maintainable Object-Oriented Software, O'Reilly Media 2020
- ▶ Erich Gamma, Ralph Johnson, Richard Helm, John Vlissides: Design Patterns: Elements of Reusable Object-Oriented Software, Addison-Wesley, 1994

MATERIALE DIDATTICO INTERNO DEL CORSO DI Ingegneria del Software.

La diffusione e pubblicazione on line del materiale è assolutamente proibita.